

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

2. Phân tích hướng Domain — DDD & Bounded Contexts	4
2.1. Conway's Law — Tổ chức quyết định kiến trúc	5
2.2. DDD cơ bản — Ngôn ngữ chung và các building blocks	7
2.2.1. Ubiquitous Language — Ngôn ngữ chung	7
2.2.2. Các building blocks	8
2.2.3. DDD dành cho kỹ sư phần mềm: Tư duy theo miền nghiệp vụ, không theo cơ sở dữ liệu	9
2.3. Strategic DDD — Bounded Context & Context Map	9
2.3.1. Bounded Context — Ranh giới ngôn ngữ	9
2.3.2. Xác định Bounded Context từ requirements	12
2.3.3. Các heuristic thực tế	12
2.3.4. Phân rã hướng dịch vụ theo Erl — Cách tiếp cận step-by-step	13
2.4. Case Study: KBM	15
2.4.1. Phân tích domain	15
2.4.2. Bầy bounded contexts core	15
2.5. Shared Kernel Pattern — Chia sẻ hay không chia sẻ?	17
2.5.1. Vấn đề thực tế	18
2.5.2. Hướng cải thiện	18
2.6. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service	20
2.7. Assemblage Process — Quy trình thiết kế service architecture	22
2.8. Tổng kết	23
2.9. Đọc thêm	23
Tài liệu tham khảo	24

2.

CHƯƠNG 2.

Phân tích hướng Domain – DDD & Bounded Contexts

“Use the model as the backbone of a language. Commit the team to exercising that language relentlessly in all communication within the team and in the code.”

— Eric Evans, *Domain-Driven Design*

MỤC TIÊU HỌC TẬP

- Hiểu Conway’s Law và tác động của cấu trúc tổ chức lên kiến trúc phần mềm
- Nắm vững các khái niệm DDD cốt lõi: Entity, Value Object, Aggregate, Repository
- Áp dụng Strategic DDD: Bounded Context, Context Map, Ubiquitous Language
- Thực hành xác định ranh giới dịch vụ từ domain — kỹ năng quan trọng nhất khi thiết kế microservices
- So sánh hai phương pháp phân tích: DDD (heuristic-based) vs Erl Service-Oriented Analysis (step-by-step)
- Phân tích case study KBM: 7 bounded contexts (5 core ban đầu + Code Practice và Examination & Proctoring) và quyết định Shared Kernel

2.1. Conway's Law — Tổ chức quyết định kiến trúc

Trước khi đi vào Domain-Driven Design, chúng ta cần hiểu một quy luật nền tảng ảnh hưởng đến mọi quyết định kiến trúc: **kiến trúc phần mềm phản ánh cấu trúc tổ chức phát triển nó.**

Quy luật Conway. Năm 1968, Melvin Conway quan sát rằng:

Conway's Law

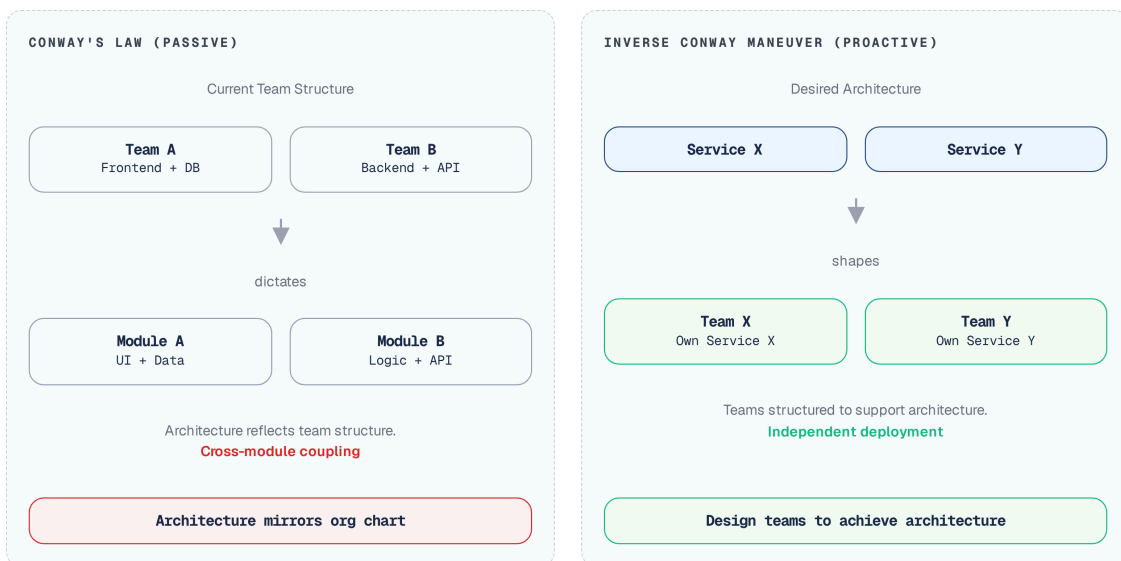
“Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure.”

— Melvin Conway, 1968

Nói cách khác: nếu tổ chức có 3 team, hệ thống sẽ có 3 service (hoặc 3 module lớn). Nếu hai nhóm phát triển giao tiếp với nhau tần suất cao, hai dịch vụ tương ứng cũng sẽ hình thành sự liên kết chặt chẽ (tight coupling). Đây không phải lý thuyết suông — nghiên cứu của MacCormack, Rusnak & Baldwin tại Harvard Business School (2008) đã cung cấp bằng chứng thực nghiệm hỗ trợ quy luật này trên các dự án mã nguồn mở.

Đối với developer và kiến trúc sư phần mềm, điều này có ý nghĩa thực tiễn rõ ràng: **không thể thiết kế microservices tốt nếu cấu trúc team không phù hợp.** Đây là lý do Amazon tái cấu trúc thành “two-pizza teams” trước khi phân rã hệ thống nguyên khối (monolithic system) (đã thảo luận ở Chương 1).

Inverse Conway Maneuver. Thay vì chấp nhận thụ động, nhiều tổ chức chủ động áp dụng Conway's Law theo hướng ngược lại: **thiết kế cấu trúc team để đạt được kiến trúc phần mềm mong muốn.** Chiến lược này gọi là *Inverse Conway Maneuver*.



Hình 2.1: Conway's Law (thụ động) vs Inverse Conway Maneuver (chủ động)

Một nhóm phát triển phụ trách một hoặc một vài dịch vụ liên quan, chịu trách nhiệm cho toàn bộ vòng đời (lifecycle) của phần mềm — từ lúc viết mã nguồn đến khi vận hành thực tế (production). Đây là mô hình “you build it, you run it” mà Amazon và Netflix đã chứng minh hiệu quả.

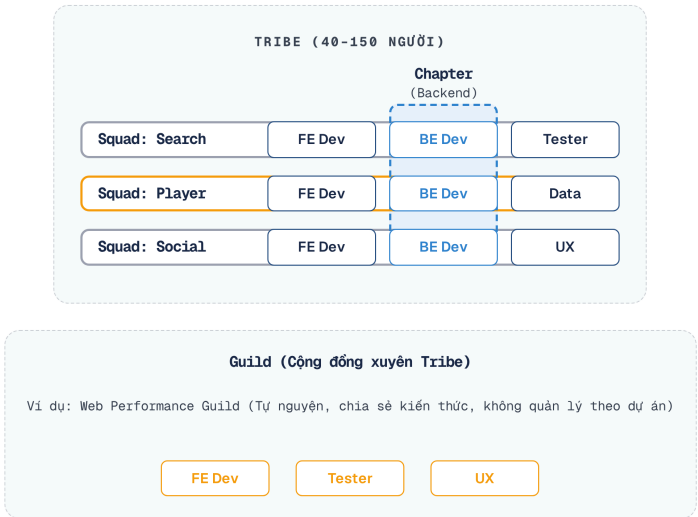
Team Topologies — Bốn kiểu team. Matthew Skelton và Manuel Pais đề xuất bốn kiểu team cơ bản cho tổ chức phần mềm hiện đại:

Kiểu team	Vai trò	Ví dụ trong KBM
Stream-aligned	Phát triển theo luồng nghiệp vụ, sở hữu end-to-end	Team “Bài tập & Chấm thi”, Team “Quản lý người dùng”
Platform	Cung cấp nền tảng nội bộ giúp stream-aligned teams phát triển nhanh hơn	Team DevOps (CI/CD, Docker, monitoring)
Enabling	Hỗ trợ các team khác nâng cao kỹ năng	Team kiến trúc (DDD coaching, code review)
Complicated subsystem	Sở hữu thành phần phức tạp đòi hỏi chuyên môn sâu	Team “SQL Judge” (sandbox isolation, multi-DBMS)

Bảng 2.1: Bốn kiểu team theo Team Topologies

Với KBM — một dự án nhỏ (2-3 lập trình viên) — không thể áp dụng đầy đủ 4 mô hình nhóm. Tuy nhiên, nguyên lý cốt lõi vẫn được giữ nguyên: lập trình viên nào am hiểu sâu sắc về một miền nghiệp vụ (domain) sẽ tự nhiên đảm nhận trách nhiệm chính cho dịch vụ tương ứng. Khi nhóm mở rộng, cấu trúc này sẽ trở nên tường minh hơn.

Industry Case Study: Spotify Squad Model. Spotify (2012) là case study nổi tiếng nhất về tổ chức team cho microservices. Henrik Kniberg và Anders Ivarsson mô tả mô hình gồm bốn cấp:



Hình 2.2: Mô hình Squad/Tribe/Chapter/Guild của Spotify

Chi tiết về cấu trúc này và ý nghĩa của chúng đối với kiến trúc vi dịch vụ được tóm tắt trong bảng sau.

Cấu trúc	Mô tả	Ý nghĩa cho microservices
Squad (6-12 người)	Đơn vị tự trị, sở hữu tính năng end-to-end	= Stream-aligned team = sở hữu 1+ microservices
Tribe (max 150 — Dunbar's number)	Nhóm squads cùng mission	Alignment mà không mất autonomy
Chapter	Nhóm người cùng competency xuyên squads	Chia sẻ thực tiễn tốt nhất (ví dụ: tất cả kỹ sư lập trình backend)
Guild	Cộng đồng quan tâm xuyên tribes	Knowledge sharing (ví dụ: Web Performance Guild)

Bảng 2.2: Cấu trúc tổ chức Spotify — ý nghĩa cho microservices

Bài học cho hệ thống nhỏ: Spotify có hơn 2.000 kỹ sư khi áp dụng mô hình này — LMS chỉ có 2-3 lập trình viên (không cần squads/tribes). Nhưng nguyên tắc cốt lõi vẫn giữ nguyên giá trị: mỗi dịch vụ cần có người chịu trách nhiệm (owner) rõ ràng, và khi nhóm mở rộng, cần tổ chức nhân sự theo các ngữ cảnh giới hạn (bounded context) (Ch.2) thay vì theo technical layer.

Nguồn: Henrik Kniberg, Anders Ivarsson, “Scaling Agile @ Spotify,” 2012. Kniberg cũng trình bày tại nhiều hội nghị (youtube.com). Lưu ý: Spotify đã tiến hóa/cải tiến mô hình này đáng kể từ năm 2012 — mô hình nguyên thủy chỉ là điểm khởi đầu, không phải là một khuôn mẫu cứng nhắc.

📌 Team nhỏ và Conway's Law

Trong LMS, cùng một developer phát triển cả Auth Service và Core Service. Điều này dẫn đến ranh giới giữa các service không rõ ràng — ví dụ: `spring.application.name` trùng nhau giữa Core và Assignment (cả hai đều là `app`), shared database vẫn được duy trì vì tiện lợi. Richardson trong [2a] cho thấy việc định nghĩa rõ service ownership từ đầu giúp tránh sự phụ thuộc ngầm (implicit coupling). **Migration path:** tách rõ naming, định nghĩa API contract giữa services, và chuẩn bị cho việc mở rộng team.

2.2. DDD cơ bản — Ngôn ngữ chung và các building blocks

Domain-Driven Design (DDD) là phương pháp thiết kế phần mềm đặt **domain** (miền nghiệp vụ) làm trung tâm. Eric Evans giới thiệu DDD năm 2003 trong cuốn sách cùng tên, và nó trở thành nền tảng cho việc phân tách microservices.

Đối với developer, DDD trả lời câu hỏi quan trọng nhất khi thiết kế microservices: **tách ở đâu?** Không phải tách theo layer kỹ thuật (UI team, DB team, API team) mà tách theo **ranh giới nghiệp vụ** — nơi mà sự thay đổi trong một domain không ảnh hưởng đến domain khác.

2.2.1. Ubiquitous Language — Ngôn ngữ chung

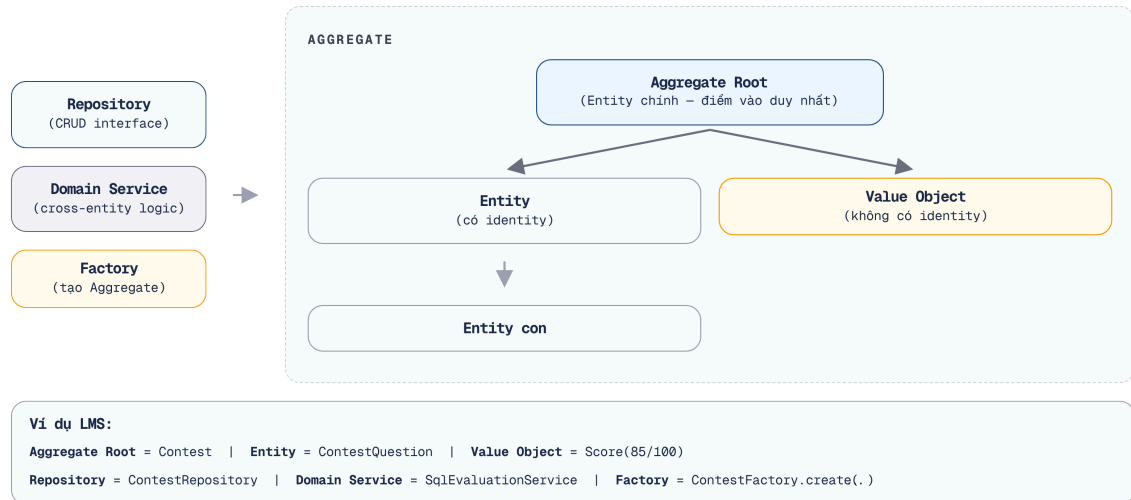
Bước đầu tiên và quan trọng nhất trong DDD không phải vẽ diagram hay viết code — mà là **xây dựng ngôn ngữ chung** (*Ubiquitous Language*) giữa developer và domain expert.

Ubiquitous Language là tập hợp thuật ngữ mà *toàn bộ nhóm phát triển* — lập trình viên, kiểm thử viên, quản lý sản phẩm — sử dụng nhất quán trong mọi giao tiếp: email, các cuộc họp, mã nguồn, tài liệu. Nếu domain expert nói “bài nộp” (*submission*), code phải có class `Submission`, database có bảng `submission`, API có endpoint `/submissions`. Không được có sự khác biệt.

Tại sao điều này quan trọng cho microservices? Vì khi hai nhóm phát triển bắt đầu dùng cùng một thuật ngữ nhưng với **ý nghĩa khác nhau**, đó chính là tín hiệu rằng họ đang ở hai bounded context khác nhau — và dịch vụ cần được phân tách.

2.2.2. Các building blocks

DDD định nghĩa một bộ building blocks để mô hình hóa domain:



Hình 2.3: Các building blocks trong DDD — Aggregate, Entity, Value Object, Repository

Entity — Đối tượng có danh tính (*identity*) duy nhất xuyên suốt vòng đời. Hai entity có cùng thuộc tính nhưng khác ID vẫn là hai entity khác nhau. Ví dụ trong KBM: `Student(id=1, name="Nguyen Van A")` và `Student(id=2, name="Nguyen Van A")` là hai sinh viên khác nhau dù trùng tên.

Value Object — Đối tượng được xác định bởi *giá trị*, không có identity riêng. Hai value object cùng giá trị là tương đương. Ví dụ: `Score(value=85, max=100)` — không quan trọng “đây là điểm nào”, chỉ quan trọng giá trị 85/100.

Aggregate — Cụm entity và value object được quản lý như một đơn vị nhất quán (*consistency boundary*). Mỗi aggregate có một **Aggregate Root** — entity duy nhất mà bên ngoài có thể tham chiếu. Trong LMS, `Contest` có thể là aggregate root bao gồm `ContestQuestion`, `ContestParticipant` — không một thành phần bên ngoài nào được phép **thay đổi dữ liệu** `ContestQuestion` mà không thông qua Aggregate Root là `Contest`. (Lưu ý: Chúng ta vẫn có thể dùng lệnh `SELECT` trực tiếp để truy vấn dữ liệu đọc nhanh, nhưng khi **Write/Ghi**, mọi logic bắt buộc phải do Root kiểm duyệt để tránh tình trạng dữ liệu không hợp lệ (data corruption)).

💡 Aggregate = Transaction boundary

Trong microservices, aggregate chính là đơn vị transaction. Một transaction chỉ nên thao tác trên **một aggregate** duy nhất. Nếu cần transaction trên nhiều aggregate, đó là tín hiệu cần Saga pattern (Chương 6). Việc sử dụng `@Transactional` để gom nhiều aggregate (nhiều bảng) vẫn có thể chấp nhận được trong Monolith, nhưng trong Microservices, nếu mở rộng một transaction xuyên suốt 2 service khác nhau sẽ làm mất khả năng mở rộng (scale) độc lập và gây khóa (deadlock) toàn hệ thống.

Repository — Interface cung cấp ảo tưởng về collection cho aggregate. Developer tương tác với repository thay vì database trực tiếp. Trong Spring Boot: `@Repository interface ContestRepository extends JpaRepository<Contest, UUID>`.

Domain Service — Logic nghiệp vụ không thuộc về một entity hay value object cụ thể nào. Ví dụ: `SqlEvaluationService` so sánh kết quả SQL của sinh viên với đáp án — logic này không thuộc về `Submission` hay `Question` riêng lẻ.

2.2.3. DDD dành cho kỹ sư phần mềm: Tu duy theo miền nghiệp vụ, không theo cơ sở dữ liệu

Sai lầm phổ biến nhất khi bắt đầu với DDD là **thiết kế entity từ bảng database**. DDD yêu cầu ngược lại: mô hình hóa domain trước, database sau.

Hai cách tiếp cận thiết kế phổ biến mang lại những kết quả hoàn toàn trái ngược nhau:

Database-first ✗ — Bắt đầu từ bảng, cột và khóa ngoại (foreign key). Kết quả thường dẫn đến entity phản ánh nguyên vẹn cấu trúc bảng, dịch vụ trở thành các lớp bọc (wrapper) đơn thuần để thao tác dữ liệu (CRUD), và ranh giới nghiệp vụ giữa các thành phần bị mờ nhạt.

Domain-first ✓ — Bắt đầu từ quy trình nghiệp vụ và ngôn ngữ chuyên ngành (domain language). Kết quả là entity phản ánh chính xác các khái niệm kinh doanh (business concepts), tạo ra những ranh giới tự nhiên và linh hoạt khi hệ thống cần mở rộng hoặc chia tách.

Trong thực tế, nhiều dự án (kể cả LMS) bắt đầu database-first. Khó khăn nghiêm trọng nhất của cách làm này là tạo ra các bảng dữ liệu tập trung khổng lồ “God Table” (ví dụ: bảng `User` phình to thành 50 cột chứa hỗn hợp nhiều loại dữ liệu từ Auth, Billing đến Profile). Khi phân rã thành các vi dịch vụ, các dịch vụ vẫn phải truy cập trực tiếp vào cùng một bảng `User` này, tạo ra sự phụ thuộc ngầm (coupling). Bước đầu tiên phải là *ngẫm lại* mô hình từ góc nhìn domain để phân rã (decompose) “God Table” đó ra.

📌 Tu duy Database-first vs Domain-first

Tưởng tượng việc xây dựng phòng Đào tạo. Nếu chúng ta nghĩ theo cách “Database-first”, chúng ta sẽ bắt đầu bằng việc mua các tủ hồ sơ, kệ sách, máy tính rồi sau đó mới nghĩ xem nhân viên sẽ làm việc thế nào với các trang thiết bị đó. Ngược lại, “Domain-first” là chúng ta vẽ ra quy trình: sinh viên nộp hồ sơ ở đâu, giáo vụ duyệt thế nào, trả kết quả ra sao, từ đó mới quyết định cần mua bao nhiêu tủ, sắp xếp bàn làm việc ra sao.

2.3. Strategic DDD — Bounded Context & Context Map

Nếu tactical DDD (Entity, Aggregate) giúp mô hình hóa *bên trong* một service, thì strategic DDD giúp trả lời câu hỏi lớn hơn: **hệ thống cần bao nhiêu service, và ranh giới ở đâu?** Để làm được điều này, DDD phân biệt rõ:

Problem Space (Không gian vấn đề) — Chia domain lớn thành các Subdomain (Subdomain Cốt lõi, Hỗ trợ, hoặc Chung).

Solution Space (Không gian giải pháp) — Ánh xạ các Subdomain này thành các Bounded Context để cài đặt bằng code (lý tưởng là 1 Subdomain = 1 Bounded Context).

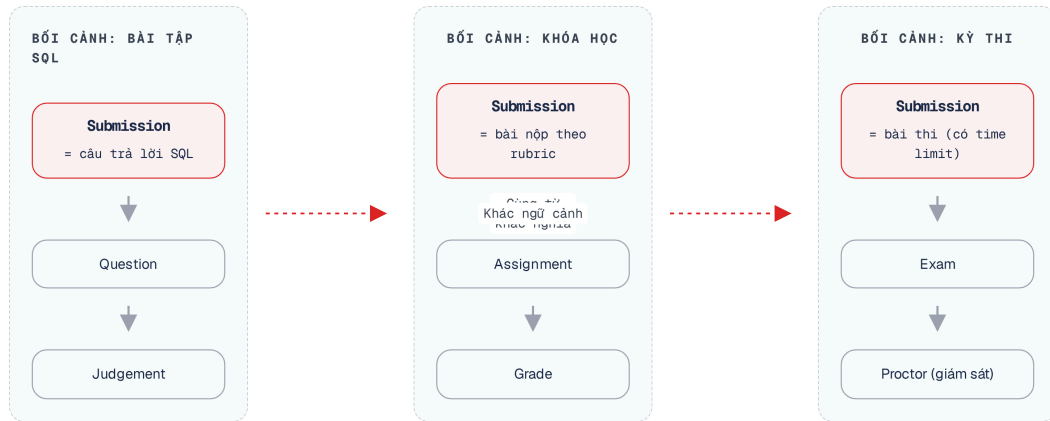
2.3.1. Bounded Context — Ranh giới ngôn ngữ

Bounded Context là khái niệm quan trọng nhất trong DDD cho microservices. Một bounded context là phạm vi trong đó một mô hình domain cụ thể có hiệu lực — và **ngôn ngữ có ý nghĩa nhất quán**. Khái niệm này có thể được ví như một Khoa trong trường đại học quản lý độc lập sinh viên và điểm số của khoa đó, không cho phép khoa khác can thiệp trực tiếp dữ liệu.

Ví dụ thực tế: trong KBM, thuật ngữ “bài nộp” (*submission*) có ý nghĩa khác nhau tùy ngữ cảnh:

- Trong context **Thực hành SQL**: submission = câu trả lời SQL của sinh viên, cần chấm đúng/sai
- Trong context **Bài tập**: submission = file/nội dung nộp bài, cần chấm điểm theo rubric
- Trong context **Thi**: submission = bài thi với giới hạn thời gian, cần kiểm tra gian lận

Cùng một từ, ba ý nghĩa khác nhau → ba bounded context tiềm năng. Đây là heuristic hiệu quả nhất để xác định ranh giới: **khi cùng một thuật ngữ mang ý nghĩa khác nhau, chúng ta đang ở ranh giới giữa hai context.**



Hình 2.4: Cùng thuật ngữ “Submission” mang ý nghĩa khác nhau trong ba bounded context

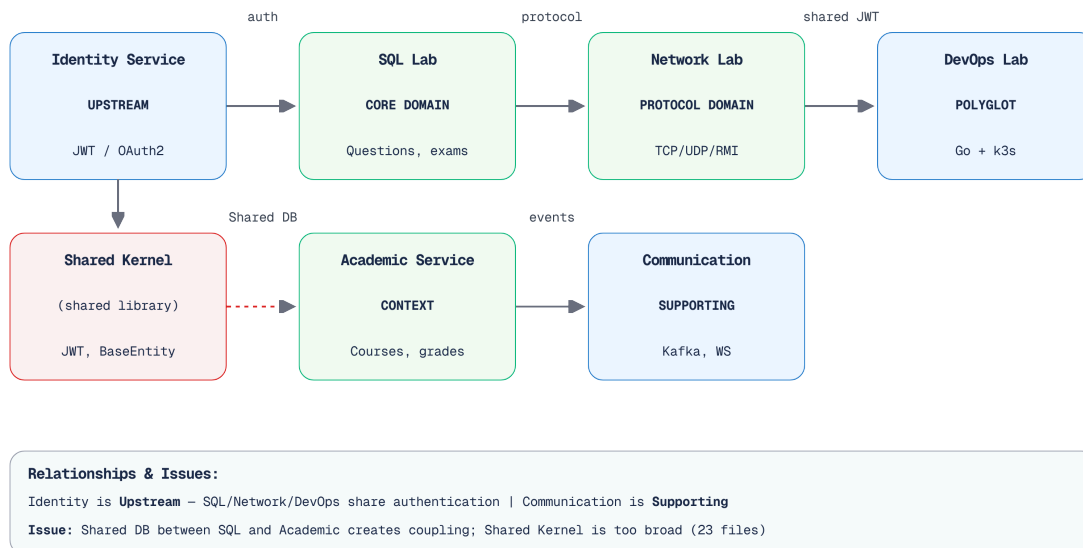
Context Map — Bản đồ quan hệ giữa các context. Khi đã xác định bounded contexts, bước tiếp theo là vẽ **Context Map** — sơ đồ quan hệ giữa chúng. Context Map không chỉ cho thấy context nào giao tiếp với nhau, mà còn cho thấy **kiểu quan hệ**.

Các kiểu quan hệ phổ biến:

Quan hệ	Mô tả	Ví dụ trong KBM
Partnership	Hai context phát triển đồng bộ, hợp tác chặt chẽ	Hai team chung dự án chia sẻ trách nhiệm
Shared Kernel	Hai context chia sẻ một phần mô hình	Shared library (base entities, JWT, exceptions)
Customer-Supplier	Nhóm Upstream (U) ưu tiên phát triển theo yêu cầu Nhóm Downstream (D)	Auth (U) → Core (D)
Conformist	(D) Hệ thống cấp khoa (Yếu) chấp nhận hoàn toàn dữ liệu từ (U) Hệ thống Quản lý Đào tạo (Mạnh)	Core phải tuân theo model của Auth cho user data
Anti-Corruption Layer	(D) Hệ thống cấp khoa tự xây bộ chuyển đổi dữ liệu từ (U) về chuẩn riêng của khoa để không bị ảnh hưởng khi (U) thay đổi	Judge Service dịch Submission thành JudgeRequest nội bộ
Open Host Service	(U) Hệ thống Quản lý Đào tạo cung cấp API chuẩn, thân thiện để hỗ trợ nhiều (D) Hệ thống của các khoa tích hợp	Core API cung cấp question data cho Judge + Assignment
Published Language	Ngôn ngữ trao đổi được chuẩn hóa (JSON, Protobuf)	Kafka messages với schema chuẩn

Bảng 2.3: Các kiểu quan hệ giữa bounded contexts

Áp dụng các mẫu quan hệ này vào KBM, chúng ta có thể vẽ ra bản đồ ngữ cảnh (Context Map) thể hiện luồng giao tiếp giữa các thành phần.



Hình 2.5: Context Map của KBM — quan hệ giữa các bounded contexts

2.3.2. Xác định Bounded Context từ requirements

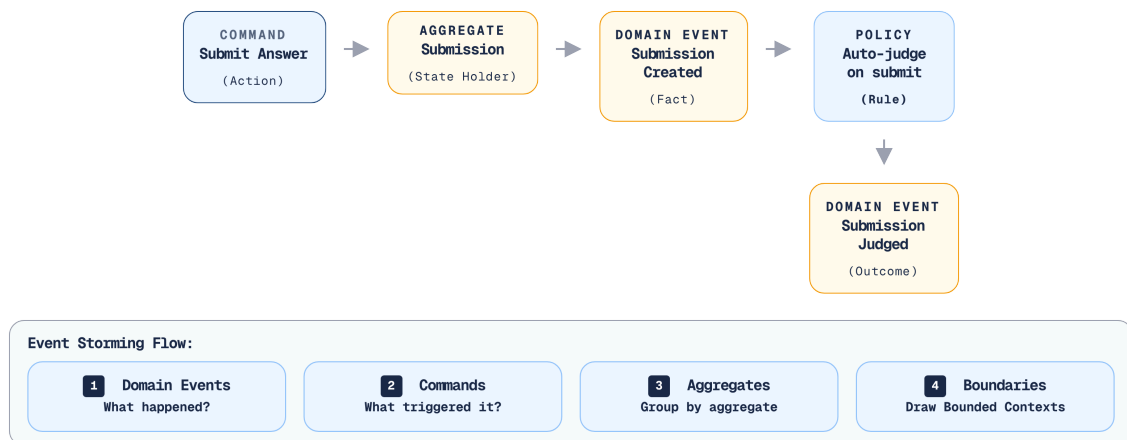
Giống như việc phân bổ ngân sách và nhân sự cho từng khoa trong trường dựa trên số lượng sinh viên đăng ký, việc xác định đúng ranh giới của các Bounded Context đòi hỏi kiến trúc sư phải dựa vào những yêu cầu nghiệp vụ thực tế thay vì cảm tính.

2.3.3. Các heuristic thực tế

Làm thế nào để “tìm” bounded context trong một hệ thống? Không có công thức chính xác — đây là kỹ năng cần luyện tập. Tuy nhiên, có một số heuristic hữu ích:

1. **Heuristic ngôn ngữ** — Khi cùng một thuật ngữ mang ý nghĩa khác nhau cho các nhóm người khác nhau, đó là ranh giới context. Ngược lại, khi hai thuật ngữ khác nhau chỉ cùng một thứ (ví dụ: “ref-des” và “component instance” trong câu chuyện PCB của Eric Evans), chúng thuộc cùng context.
2. **Heuristic thay đổi** — Tự hỏi: “Khi yêu cầu nghiệp vụ X thay đổi, những đoạn mã nguồn nào cần sửa?” Nếu thay đổi luôn ảnh hưởng đến cùng một nhóm tệp tin (files), chúng thuộc cùng context. Nếu thay đổi lan sang nhóm khác, đó là tín hiệu coupling cần xem xét.
3. **Heuristic team** — Bộ phận nào sẽ đảm nhận phát triển và bảo trì phần này? Nếu do cùng một nhóm phụ trách, các thành phần này *có thể* thuộc cùng một ngữ cảnh. Nếu do các nhóm khác nhau đảm nhận, cần *cân nhắc* việc phân tách ngữ cảnh (Conway’s Law).
4. **Heuristic data** — Dữ liệu nào “thuộc về” nhau? Entity nào *luôn* được truy vấn cùng nhau? Đó là aggregate trong cùng context. Entity nào chỉ được tham chiếu qua ID? Đó có thể thuộc context khác.

Event Storming — Phương pháp khám phá domain. Event Storming là workshop technique do Alberto Brandolini phát triển, giúp cả team cùng khám phá domain thông qua *domain events* — những sự kiện đã xảy ra trong hệ thống.



Hình 2.6: Event Storming — luồng từ Command đến Domain Event

Quy trình cơ bản:

1. **Liệt kê domain events** — giấy ghi chú (post-it) màu cam: “Submission Created”, “SQL Executed”, “Score Calculated”
2. **Tìm commands** — giấy ghi chú màu xanh: “Submit Answer”, “Start Exam”, “Create Assignment”
3. **Nhóm events theo aggregate** — giấy ghi chú màu vàng: events nào cùng xảy ra trên cùng entity?
4. **Vẽ ranh giới** — nhóm aggregates có liên quan chặt chẽ → bounded context

Kết quả Event Storming tự nhiên dẫn đến bounded contexts. Nhóm events/aggregates nào tương tác nhiều nội bộ nhưng ít tương tác với nhóm khác → đó là context.

2.3.4. Phân rã hướng dịch vụ theo Erl — Cách tiếp cận step-by-step

DDD và Event Storming là phương pháp *khám phá* — bắt đầu từ ngôn ngữ domain, dần dần tìm ra ranh giới. Cách tiếp cận này mạnh nhưng đòi hỏi kinh nghiệm: các lập trình viên thiếu kinh nghiệm thường gặp khó khăn trong việc xác định điểm dừng hoặc tính đúng đắn của ranh giới.

Thomas Erl trong *SOA: Analysis and Design for Services and Microservices* [1, Ch.8–10] đề xuất một phương pháp hỗ trợ: **Service-Oriented Analysis** — quy trình *prescriptive*, gồm các bước cụ thể từ yêu cầu nghiệp vụ đến danh mục dịch vụ. Đây là cách tiếp cận dễ dạy và dễ học hơn cho sinh viên, vì mỗi bước có input và output rõ ràng.

Năm bước phân rã dịch vụ theo Erl:

1. **Xác định automation candidates** — Phân tích quy trình nghiệp vụ (business process) và xác định những bước nào *nên* được tự động hóa bằng phần mềm. Erl gọi đây là *service candidates* — chưa phải services, mà là ứng viên.

Ví dụ KBM: quy trình “Sinh viên nộp bài SQL” có các bước: (1) xác thực người dùng, (2) nhận câu trả lời, (3) thực thi SQL trên sandbox, (4) so sánh kết quả, (5) ghi điểm, (6) thông báo. Mỗi bước là một automation candidate.

1. **Phân loại service layers** — Erl định nghĩa ba loại service:

Ba loại service theo đề xuất của Erl bao gồm:

Task Service – Chịu trách nhiệm điều phối quy trình nghiệp vụ (workflow) bằng cách gọi các service khác. Ví dụ trong KBM: Submission orchestrator xử lý chuỗi hành động bao gồm nhận bài → gọi dịch vụ chấm điểm (Judge) → ghi điểm → thông báo.

Entity Service – Chuyên quản lý dữ liệu nghiệp vụ bao gồm các thao tác CRUD và quy tắc kinh doanh (business rules). Ví dụ: Question Service, User Service, Contest Service.

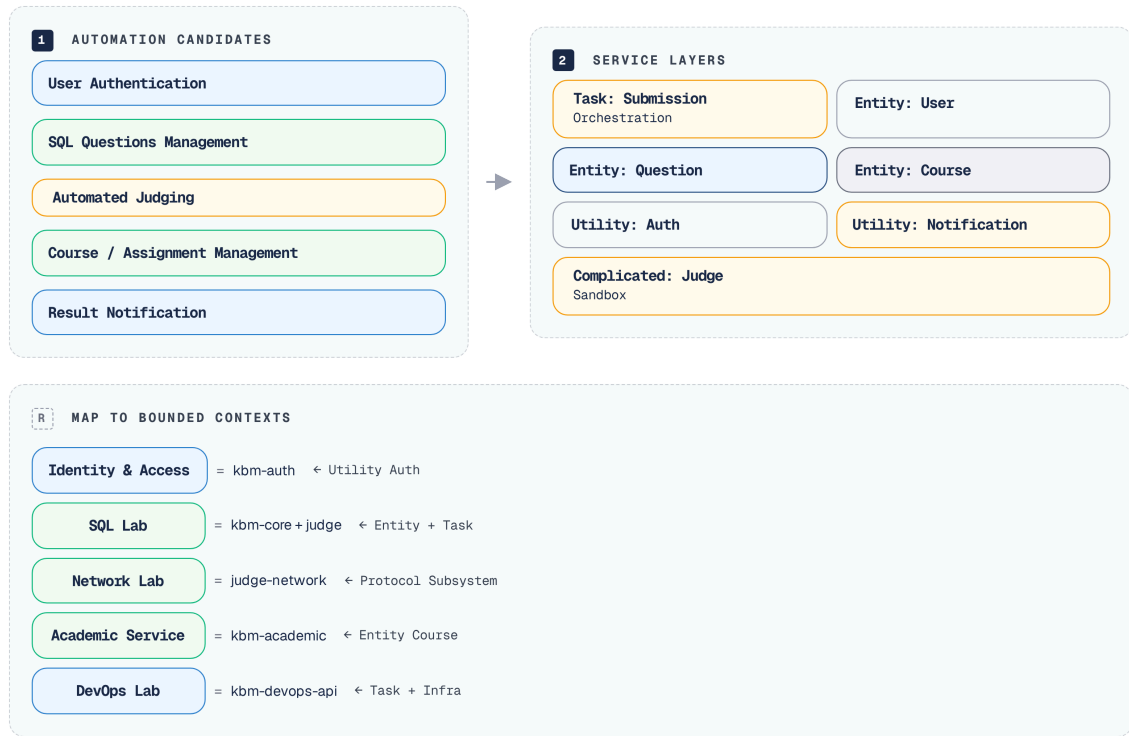
Utility Service – Cung cấp các chức năng kỹ thuật được dùng chung trên toàn hệ thống. Ví dụ: Dịch vụ kiểm tra JWT (validation), gửi thông báo (notification), hoặc ghi nhật ký hệ thống (logging).

Phân loại này giúp tránh sai lầm phổ biến: tạo ra các dịch vụ chỉ đơn thuần thực hiện CRUD (entity service không có business logic) hoặc gom toàn bộ logic vào một “god service” (thiếu task/utility decomposition).

1. **Tạo service inventory blueprint** — Vẽ bản đồ toàn bộ services dự kiến, mỗi dịch vụ với các thao tác (operations) chính. Tương tự Context Map trong DDD, nhưng tập trung hơn vào các năng lực (*capabilities*) thay vì *language boundaries* (ranh giới ngôn ngữ).
2. **Xác định service boundaries + composition** — Kiểm tra từng service candidate: nó có thể hoạt động **độc lập** không? Nếu dịch vụ A *luôn* cần gọi dịch vụ B trước khi có thể xử lý bất kỳ yêu cầu (request) nào → cân nhắc việc gộp chung. Nếu dịch vụ A vẫn có thể hoạt động độc lập ngay cả khi dịch vụ B gặp sự cố (ngừng hoạt động) → ranh giới tốt.
3. **Verify qua 8 nguyên lý SOA** — Mỗi service phải thỏa mãn 8 nguyên lý hướng dịch vụ đã thảo luận ở Bảng 1.2 (Chương 1): Standardized Contract, Loose Coupling, Abstraction, Reusability, Autonomy, Statelessness, Discoverability, Composability. Đây là “checklist kiểm tra chất lượng” cho service design.

Áp dụng Erl cho KBM:

Áp dụng 5 bước trên cho KBM:



Hình 2.7: Áp dụng Erl's Service-Oriented Analysis cho KBM — từ automation candidates đến service layers

Kết quả: 5 nhóm dịch vụ — gần như trùng khớp với 5 bounded contexts từ phân tích DDD (§2.5): Identity & Access $\approx$ Utility Auth, SQL Practice $\approx$ Entity Question + Task Orchestrator, Network Practice $\approx$ Complicated Protocol Subsystem, Academic $\approx$ Entity Course, DevOps Practice $\approx$ Task Service + Infrastructure Automation.

So sánh: Erl vs DDD.

Tiêu chí	Erl Service-Oriented Analysis	DDD Bounded Context
Điểm bắt đầu	Quy trình nghiệp vụ (business process)	Ngôn ngữ domain (Ubiquitous Language)
Phong cách	Prescriptive — 5 bước rõ ràng, input/output mỗi bước	Explorative — heuristics, Event Storming workshop
Tiêu chí tách	Service layers (task/entity/utility) + 8 nguyên lý SOA	Ranh giới ngôn ngữ + aggregate consistency
Output	Service inventory blueprint	Context Map + Bounded Context diagram
Thế mạnh	Systematic, dễ dạy, phù hợp team mới	Sâu về domain, phát hiện ranh giới ẩn
Hạn chế	Có thể tạo ranh giới quá kỹ thuật (thiếu domain insight)	Cần kinh nghiệm, kết quả phụ thuộc người phân tích
Phù hợp khi	Team ít kinh nghiệm DDD, cần quy trình chuẩn	Team hiểu domain sâu, cần phát hiện complexity ẩn

Bảng 2.4: Hai phương pháp phân tách service — Erl Service-Oriented Analysis vs DDD

Dù tiếp cận theo cách nào, mục tiêu cuối cùng vẫn là tìm ra ranh giới hợp lý nhất cho hệ thống. Thay vì coi đây là hai trường phái đối lập, hãy xem chúng là các công cụ hỗ trợ lẫn nhau.

 **Bổ trợ, không thay thế**

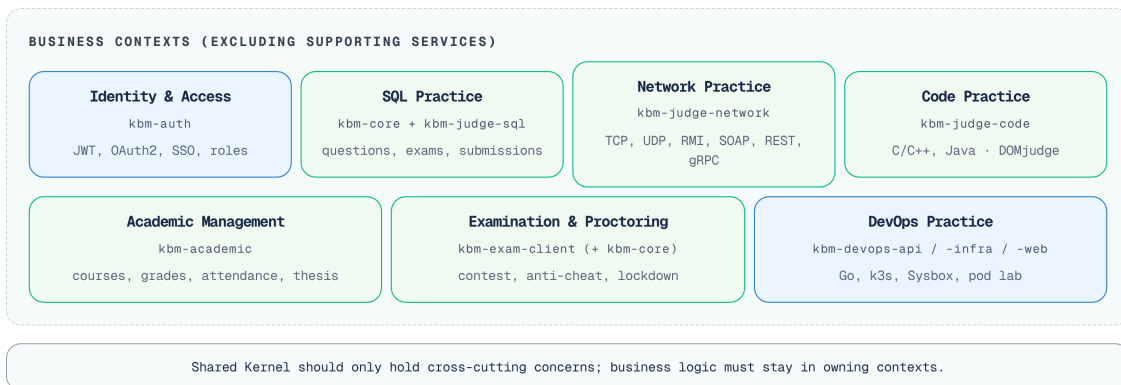
Erl cung cấp *quy trình* (process) — đặc biệt hữu ích khi chúng ta chưa biết bắt đầu từ đâu. DDD cung cấp *insight* — phát hiện những ranh giới mà quy trình cơ học bỏ sót. Trong thực tế: dùng Erl để tạo bản phác thảo ban đầu (service inventory), sau đó dùng DDD (Event Storming, ngôn ngữ chung) để *tinh chỉnh* ranh giới. Richardson's Assemblage Process (§2.7) kết hợp cả hai hướng.

2.4. Case Study: KBM

Trở lại với bài toán hệ thống LMS của trường đại học, chúng ta không thể giao toàn bộ việc quản lý điểm số, học phí và lịch thi cho cùng một bộ phận. Việc chia nhỏ thành các Bounded Context giúp định hình rõ quyền hạn và trách nhiệm của từng phòng ban.

2.4.1. Phân tích domain

KBM phục vụ các hoạt động giảng dạy, thực hành, đánh giá và vận hành lab thực hành. Từ góc nhìn DDD, chúng ta phân tách Problem Space thành các Subdomain: SQL Practice là Core Domain (cốt lõi tạo ra giá trị), Auth là Generic Subdomain (đã có chuẩn chung ngoài thị trường), Academic Management là Supporting Subdomain (cần thiết nhưng không cốt lõi). Còn các service như Gateway, Registry là Infrastructure (hạ tầng kỹ thuật), tuyệt đối không nhầm lẫn chúng với Subdomain.



Hình 2.8: Các bounded context của KBM (năm context core ban đầu; Code Practice và Examination & Proctoring được bổ sung khi hệ thống tiến hóa)

2.4.2. Bảy bounded contexts core

1. Identity & Access

(**kbm-auth**)

Domain rõ ràng nhất — quản lý danh tính người dùng, xác thực, phân quyền. Entities: **User** , **Role** , **Token** . Đây là context upstream — mọi context khác đều phụ thuộc vào kết quả authentication.

Trong KBM, context này được tách thành service riêng sớm nhất (**kbm-auth**), vì ranh giới rất rõ: thay đổi cơ chế xác thực (tài khoản nội bộ, Google/Microsoft OAuth2, email verification, SSO từ hệ thống đào tạo) không ảnh hưởng logic bài tập hay chấm thi.

2. SQL Practice

(`kbm-core` + `kbm-judge-sql` — *Core Domain*)

Đây là **core domain** ban đầu — phần tạo ra giá trị chính cho hệ thống SQL judge. Entities: `Question` , `Submission` , `Contest` , `UserContest` , `Statistic` . Toàn bộ quy trình từ “sinh viên xem câu hỏi → viết SQL → nộp bài → nhận kết quả” nằm trong context này.

Đây là context lớn nhất và phức tạp nhất. Bên trong nó có kiến trúc chấm SQL gồm một service điều phối `judge` và nhiều processor `judge-*` theo DBMS (`judge-mysql` , `judge-sqlserver` , ...). Đây mới là nơi cần phân tích kỹ về các vấn đề định tuyến (routing), khả năng phục hồi (resilience), tính lũy đẳng (idempotency) và quyền sở hữu (ownership) giữa thành phần điều phối (coordinator) và thành phần xử lý (worker); không nên nhầm với Network Practice.

3. Network Practice

(`kbm-judge-network`)

Domain chuyên biệt cho bài thực hành lập trình mạng Java: TCP, UDP, RMI, SOAP, REST và gRPC. Khác với SQL Practice, sinh viên viết client trên IDE cá nhân và kết nối trực tiếp đến judge server. Context này minh họa ranh giới domain đến từ **protocol semantics**, không chỉ từ entity/database.

Network Practice được thiết kế khá độc lập với các services còn lại: nó phục vụ mục tiêu đánh giá giao tiếp qua mạng, có protocol contract riêng và không cần chia sẻ pipeline chấm SQL. Trong sách, context này chủ yếu minh họa **technology diversity** và boundary theo protocol, không phải một *gap* kiến trúc cần sửa như SQL Judge.

4. Academic Management

(`kbm-academic`)

Quản lý khóa học, bài tập dạng rubric, đăng ký môn, điểm số, điểm danh, MCQ daily, syllabus/CLO, GitHub Classroom và đồ án tốt nghiệp. Entities: `Course` , `Assignment` , `Enrollment` , `Grade` , `AttendanceSession` , `Thesis` .

Context này được tách ra từ Core Service — nhưng *vẫn sử dụng chung cơ sở dữ liệu* (`app_db`). Đây là trade-off quan trọng mà chúng ta sẽ phân tích trong phần tiếp theo.

5. DevOps Practice

(`kbm-devops-api` , `kbm-devops-infra` , `kbm-devops-web`)

Domain mới phục vụ bài thực hành Docker/Kubernetes: tạo lab environment cô lập, terminal web, validator script và progress tracking. Context này dùng Go, k3s và Sysbox, sử dụng stack công nghệ khác với LMS chính. Đây là ví dụ tốt cho polyglot microservices: dùng công nghệ khác khi domain và hạ tầng yêu cầu, nhưng vẫn chia sẻ danh tính (identity) qua JWT của KBM.

6. Code Practice

(`kbm-judge-code`)

Domain chấm bài lập trình thuật toán đa ngôn ngữ (C/C++, Java, Python...) theo mô hình ICPC: sinh viên nộp mã nguồn, hệ thống biên dịch, chạy với bộ test, đối chiếu đầu ra (output) và trả về kết quả (Verdict). Trong giai đoạn đầu, hệ thống sử dụng một bộ chấm điểm *tự phát triển nội bộ* (chỉ C/C++ và Java, không sandbox); về sau được thay bằng `kbm-judge-code` tích hợp **DOMjudge** (hệ thống chấm bài trực tuyến mã nguồn mở chuẩn thi đấu). Context này tách khỏi SQL Practice vì *mô hình chấm khác hẳn*: SQL so sánh result set trên CSDL thật, còn Code Practice biên dịch–chạy–so sánh kết quả đầu ra (output) trong sandbox cô lập. Đây cũng là minh họa kinh điển cho quyết định **build vs buy** (sẽ phân tích ở Ch.4, Ch.10).

7. Examination & Proctoring

(logic Contest/Anti-cheat hiện trong `kbm-core` , mục tiêu tách `kbm-exam` ; `kbm-exam-client`)

Domain thi cử và giám sát: vòng đời Contest (Scheduled → Open → Closed), bảng xếp hạng thời gian thực, chống gian lận (chuyển thẻ trình duyệt (tab), sao chép-dán (copy-paste), đổi IP/User-Agent) và đặc biệt là kỳ thi **trên phòng máy** qua ứng dụng desktop `kbm-exam-client` (lockdown toàn màn hình, whitelist IP phòng thi, giám sát từ phía máy khách (client-side proctoring)). Về mặt nghiệp vụ đây là một bounded context riêng, dù *hiện tại* phần điều phối thi vẫn nằm chung trong `kbm-core` (một biểu hiện của siêu dịch vụ ôm đồm - god-service) — mục tiêu chuyển đổi (migration target) hệ thống là phân tách thành `kbm-exam`. Context này minh họa nguyên tắc **một backend phục vụ nhiều loại client** (web luyện tập + desktop thi) và **defense-in-depth** cho anti-cheat (Ch.9).

Từ bounded context đến service.

Bounded Context	Service	Lý do tách
Identity & Access	<code>kbm-auth</code>	Ranh giới rõ ràng, thay đổi auth ≠ thay đổi logic học tập
SQL Practice	<code>kbm-core</code> , <code>kbm-judge-sql</code> , sandbox DB services	Core domain, cần sandbox isolation và chấm tự động
Network Practice	<code>kbm-judge-network</code>	Protocol đa dạng, state/routing khác REST API thông thường
Code Practice	<code>kbm-judge-code</code>	Mô hình chấm compile-run-compare khác SQL; tích hợp DOMjudge (build vs buy)
Academic Management	<code>kbm-academic</code>	Domain học vụ, grade scheme, attendance, thesis
Examination & Proctoring	<code>kbm-exam-client</code> (+ logic trong <code>kbm-core</code> , target <code>kbm-exam</code>)	Thi cử, lockdown phòng máy, anti-cheat; nhiều loại client một backend
DevOps Practice	<code>kbm-devops-api</code> , <code>kbm-devops-infra</code> , <code>kbm-devops-web</code>	Domain lab infrastructure, Go/k3s/Sysbox, validator script

Bảng 2.5: Ánh xạ bounded context → service trong KBM

Khi đối chiếu bảng phân rã này, một điểm quan trọng cần lưu ý là mối quan hệ giữa ranh giới nghiệp vụ và cấu trúc triển khai vật lý.

💡 Bounded context ≠ service

Bounded context là ranh giới logic. Một ngữ cảnh có thể được hiện thực hóa (implement) bởi nhiều dịch vụ (như SQL Practice = Core + Judge + sandbox DB services). Ngược lại, một kho mã nguồn (repository) cũng có thể tạo ra nhiều tệp thực thi (binary) phục vụ cùng context (như DevOps Practice = API + router + grader trong một codebase Go) — miễn là ranh giới domain rõ ràng.

2.5. Shared Kernel Pattern — Chia sẻ hay không chia sẻ?

Trong môi trường đại học, có những thông tin bắt buộc phải dùng chung như mã sinh viên hay quy chế đào tạo cốt lõi. Tương tự, Shared Kernel là phần mã nguồn hoặc mô hình dữ liệu mà nhiều Bounded Context buộc phải đồng thuận chia sẻ, dù điều này có thể làm giảm tính độc lập.

2.5.1. Vấn đề thực tế

Trong KBM, tồn tại một **shared library** chứa 23 Java files: base entities, DTOs, JWT utilities, exception handling, validation utilities. Nhiều service Java phụ thuộc vào library này.

Đây là pattern **Shared Kernel** trong DDD: hai hoặc nhiều bounded contexts đồng ý chia sẻ một phần mô hình chung. Shared Kernel giúp giảm thiểu sự trùng lặp (duplication) nhưng lại sinh ra sự phụ thuộc (coupling) — mọi thay đổi trong thư viện dùng chung (shared library) đều *có thể* ảnh hưởng đến tất cả các dịch vụ. Cần phân biệt rõ: chia sẻ mã nguồn hạ tầng (JWT, CORS) chỉ là pattern **Shared Library** thông thường. Thuật ngữ **Shared Kernel** trong DDD đặc biệt chỉ việc chia sẻ các khái niệm cốt lõi của **Domain Model** (như các Enums, BaseEntity). Trong microservices, chúng ta khuyến khích Shared Library cho hạ tầng, nhưng hạn chế tối đa Shared Kernel cho domain để tránh coupling.

```

kbm-shared/
├── common/      BaseMapper, BaseRequest, BaseResponse, BaseService
├── config/      AuditConfig, CorsConfig, TimeZoneConfig
├── enumerate/   AnswerStatus, ErrorCode, TypeQuestion
├── exception/   GlobalExceptionHandler
├── securityConfig/ JwtConfig, UserDetailCustom
└── utils/      CompareUtil, JwtUtil, SqlUtil, TableDependencyResolver
  
```

Phân tích: cái gì nên shared, cái gì không?

Loại	Ví dụ	Nên shared?	Lý do
Cross-cutting concerns	JWT validation, exception handling, CORS config	✓ Có	Nhất quán hành vi giữa các dịch vụ, ít thay đổi
Base abstractions	BaseEntity, BaseMapper, BaseResponse	! Cần nhắc	Mang lại sự tiện lợi nhưng tạo ra sự phụ thuộc (coupling) cũng nhắc vào các quy ước chung (convention)
Domain-specific logic	<code>CompareUtil</code> (SQL comparison), <code>SqlUtil</code>	× Không	Chỉ Judge context cần — không phải shared concern
Domain enums	<code>TypeQuestion</code> , <code>AnswerStatus</code>	× Không	Thuộc về SQL Practice context, không universal

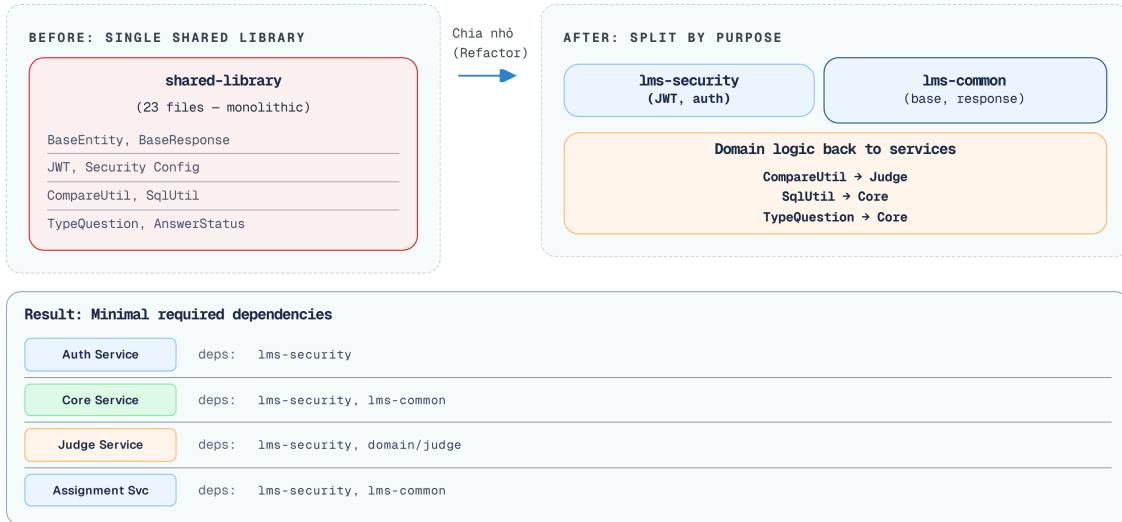
Bảng 2.6: Phân tích shared library — cái gì nên shared, cái gì không

2.5.2. Hướng cải thiện

Nếu team phát triển lớn hơn và cần giảm coupling, shared library có thể được tách thành:

1. **kbm-security** — JWT, cấu hình xác thực (mọi dịch vụ đều cần)
2. **kbm-shared** — Base entities, response format (convention)
3. **Domain-specific code** → chuyển vào service tương ứng (Judge, Core)

Chiến lược tái cấu trúc này được minh họa cụ thể trong sơ đồ dưới đây.



Hình 2.9: Hướng tách shared library — từ monolithic sang modular

Việc tái cấu trúc thư viện dùng chung như được minh họa ở trên giúp giảm bớt sự phụ thuộc không cần thiết giữa các module.

Shared Kernel Trade-off

Shared Kernel là *thỏa thuận có chủ đích*, không phải sự tiện lợi. Mỗi file trong shared library phải trả lời: “Nếu thay đổi file này, tất cả service *phải* thay đổi cùng nhau — và điều đó *hợp lý*.” Nếu câu trả lời là không, file đó không nên nằm trong shared kernel.

Shared Kernel cần tách

Với 23 files trong shared library, KBM đang tạo ra sự ràng buộc (coupling) không cần thiết: domain-specific code như `CompareUtil` nằm chung với các mối quan tâm xuyên suốt (cross-cutting concerns) như JWT. Richardson khuyến nghị chỉ chia sẻ những thành phần thực sự mang tính xuyên suốt (cross-cutting) (event definitions, API contracts). **Migration path:** (1) tách domain code về service riêng, (2) giữ shared chỉ security + base response, (3) đưa domain enums (`TypeQuestion` , `AnswerStatus`) về context sở hữu.

Tuy nhiên, cần tránh những lầm tưởng kinh điển sau:

⚠ Sai lầm thường gặp

1. **Phân rã theo schema database** — Thiết kế service dựa trên các bảng CSDL thay vì nghiệp vụ. Hậu quả: tạo ra các CRUD API mỏng. *Phòng tránh*: Bắt đầu từ domain process (Event Storming §2.4), không từ schema.
2. **Chia quá nhỏ (nano-services)** — Tách mỗi entity thành 1 service riêng (UserService, RoleService, TokenService) khi chúng luôn thay đổi cùng nhau. Hậu quả: mỗi yêu cầu (request) cần gọi chéo 3-4 dịch vụ, dẫn đến độ trễ (latency) tăng cao và quá trình gỡ lỗi (debugging) trở nên vô cùng phức tạp. *Phòng tránh*: tách theo bounded context, không theo entity. Nếu hai entity luôn thay đổi cùng nhau, chúng thuộc cùng service.
3. **Dùng Shared Kernel vì tiện, không vì thiết kế** — Đưa toàn bộ mã nguồn chung vào thư viện dùng chung mà không phân biệt cross-cutting concern và domain logic. Hậu quả: mỗi thay đổi trong thư viện dùng chung sẽ ảnh hưởng đến tất cả các dịch vụ, tạo ra sự phụ thuộc ngầm khi triển khai (deployment coupling). *Phòng tránh*: mỗi file trong shared kernel phải trả lời “tất cả service *phải* thay đổi cùng khi file này thay đổi?” (§2.6).

2.6. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service

(Mục nâng cao — người đọc mới có thể đọc lướt lần đầu. Cốt lõi cần nhớ là cặp “lực đẩy / lực hút”; chi tiết 10 lực dùng khi thực sự phải quyết định tách/gộp một service cụ thể.)

Richardson trong phiên bản thứ hai (2025) giới thiệu một framework phân tích trade-off cho việc service decomposition, mượn ẩn dụ từ vật lý thiên văn:

Dark Energy – lực **đẩy** (repulsive): kéo các thành phần (components) ra xa nhau → nên phân tách thành dịch vụ riêng biệt

Dark Matter – lực **hút** (attractive): kéo các components lại gần nhau → nên giữ cùng service

Lực	Loại	Mô tả	Ví dụ KBM
Simple interactions	● Matter (hút)	Components tương tác phức tạp → giữ cùng service	ContestQuestion ↔ Contest luôn query chung → hợp lý ở cùng Core Service
Efficient interactions	● Matter (hút)	Yêu cầu độ trễ (latency) thấp → việc gọi hàm nội bộ (in-process) luôn nhanh hơn gọi qua mạng	Submission → Score cập nhật cần nhanh → hiện cùng service
Prefer ACID	● Matter (hút)	Cần ACID transaction → cùng DB dễ hơn saga	Create contest + add questions = 1 transaction → cùng service
Minimize runtime coupling	● Matter (hút)	Giảm thiểu sự phụ thuộc trong thời gian chạy (runtime dependency) dẫn đến giảm số lượng lệnh gọi qua mạng	Judge Service xử lý độc lập (nhận tin nhắn → thực thi → trả kết quả)
Simple components	[Energy] (đẩy)	Mỗi service nhỏ, dễ hiểu	Core Service (24 files) vs nếu gộp tất cả (50+ files)
Team autonomy	[Energy] (đẩy)	Các nhóm triển khai (deploy) độc lập, không cần phối hợp chéo (coordination)	Nhóm phát triển Auth và Core có thể triển khai (deploy) độc lập
Fast deployment pipeline	[Energy] (đẩy)	Dịch vụ nhỏ gọn giúp quá trình biên dịch (build) và kiểm thử (test) diễn ra nhanh chóng	Auth Service build 30s vs monolith build 5 phút
Support multiple tech stacks	[Energy] (đẩy)	Các dịch vụ khác nhau có thể sử dụng nền tảng công nghệ (tech stack) khác nhau	Judge Service có thể viết bằng Go (performance)
Independent scalability	[Energy] (đẩy)	Mở rộng quy mô (scale) độc lập cho từng dịch vụ theo nhu cầu	Judge Service cần scale khi contest, Auth thì không
Independent data ownership	[Energy] (đẩy)	Mỗi dịch vụ làm chủ dữ liệu riêng, đảm bảo tính đóng gói (encapsulation)	Users ở Auth, Questions ở Core (lý tưởng)

Bảng 2.7: Mười lực Dark Energy / Dark Matter

Cách sử dụng: Khi phân vân “có nên tách component X thành service riêng?”, liệt kê các lực tác động. Nếu Dark Energy (đẩy) chiếm ưu thế → tách. Nếu Dark Matter (hút) mạnh hơn → giữ chung. Đây không phải công thức chính xác mà là **framework tư duy** giúp cấu trúc cuộc thảo luận.

Dark Energy/Matter cho quyết định tách Core & Assignment

KBM hiện có Core Service và Assignment Service dùng chung cơ sở dữ liệu. Phân tích:

Dark Matter (giữ chung) – cơ sở dữ liệu dùng chung giúp đảm bảo ACID dễ dàng, bảng `assignment_questions` tham chiếu tới `questions.id` (tương tác đơn giản), nhóm phát triển rất nhỏ (2-3 người)

Dark Energy (tách) – domain logic khác biệt (bài tập vs cuộc thi), data ownership rõ ràng, potential for independent deployment

Kết luận: Dark Matter hiện mạnh hơn (do team nhỏ, shared DB). Khi quy mô nhóm phát triển vượt mức 5 thành viên, trạng thái cân bằng sẽ dịch chuyển → lúc đó việc phân tách cơ sở dữ liệu và dịch vụ sẽ trở nên hợp lý hơn. Đây chính là nguyên tắc “*Don’t decompose prematurely*” (Richardson [2b, Ch.7]).

2.7. Assemblage Process – Quy trình thiết kế service architecture

(Mục nâng cao — tổng hợp các kỹ thuật phía trên thành một quy trình. Người đọc mới nên nắm chắc Bounded Context và Context Map trước; có thể quay lại quy trình tổng hợp này khi bắt tay thiết kế một hệ thống thật.)

Làm thế nào để tổng hợp tất cả các kỹ thuật trên — Bounded Context, Context Map, Dark Energy/Matter — thành một quy trình thiết kế mang tính thực hành? Richardson trong [2b, Ch.20] trình bày **Assemblage Process**: quy trình step-by-step để đi từ yêu cầu nghiệp vụ đến kiến trúc microservices cụ thể.

1. **Định nghĩa System Operations** — Liệt kê các thao tác chính mà hệ thống phải hỗ trợ (ví dụ: `submitAnswer()`, `createContest()`, `gradeSubmission()`).
2. **Xác định Subdomains** — Dùng Bounded Contexts (§2.3) để phân vùng nghiệp vụ: Identity, SQL Practice, Network Practice, Academic Management, DevOps Practice.
3. **Gán Operations → Subdomains** — Quyết định logic nghiệp vụ của mỗi operation thuộc subdomain nào.
4. **Áp dụng Dark Energy & Dark Matter** — Với mỗi cặp subdomains, phân tích 10 lực đã thảo luận ở phần trước: lực đẩy (Dark Energy) khuyến khích tách thành service độc lập; lực hút (Dark Matter) giữ các phần chặt chẽ lại cùng nhau.
5. **Thiết kế IPC** — Chọn mô hình giao tiếp giữa các service: đồng bộ REST/gRPC (Ch.4) hay bất đồng bộ Kafka (Ch.5).
6. **Lặp lại** — Quy trình không chạy một lần. Khi kiến thức domain sâu hơn, ranh giới service cần được đánh giá lại.

Assemblage process biến quá trình thiết kế kiến trúc — vốn mang tính nghệ thuật và chủ quan — thành chuỗi các quyết định kỹ thuật có thể giải thích được. Thay vì nói “tách Judge Service vì có vẻ hợp lý”, chúng ta có thể lập luận: “*Judge Service được tách vì Dark Energy force #1 (simple components) và #3 (fast deployment pipeline) vượt trội so với Dark Matter force #2 (efficient interaction) trong context này.*”

Interactive Demo

Xem minh họa động về **Bản đồ Bounded Context (Context Map)** tại companion web: context-map.html

2.8. Tổng kết

Quy trình phân tách hệ thống trải dài từ cấp vĩ mô (Conway’s Law) đến vi mô (Entity, Aggregate, Repository).

Conway’s Law nhắc nhở rằng kiến trúc phần mềm không tồn tại trong chân không — nó phản ánh và bị ràng buộc bởi cấu trúc tổ chức. Inverse Conway Maneuver cho phép chúng ta chủ động thiết kế team để đạt kiến trúc mong muốn.

Chúng ta có hai phương pháp hỗ trợ để xác định ranh giới service. **DDD** cung cấp ngôn ngữ và heuristics — Bounded Context, Context Map, Event Storming — phù hợp khi team hiểu domain sâu. **Erl’s Service-Oriented Analysis** cung cấp quy trình step-by-step — từ automation candidates đến service layers — phù hợp khi team cần một framework có cấu trúc. Cả hai đều dẫn đến kết quả tương tự (như minh họa với KBM: 5 service groups chính), nhưng từ góc nhìn khác nhau. Dark Energy/Dark Matter forces lượng hóa các yếu tố ảnh hưởng đến quyết định tách/gộp, và Assemblage Process (§2.7) tổng hợp tất cả thành quy trình thiết kế có hệ thống.

Phân tích KBM cho thấy 5 bounded contexts rõ ràng, mỗi context tương ứng với một nhóm service hoặc binary. Shared Kernel — dù tiện lợi cho team nhỏ — cần được quản lý có chủ đích, phân biệt rõ giữa cross-cutting concerns (nên share) và domain logic (không nên share).

Bài tập Chương 2

Xem Phụ lục Bài tập để luyện tập:

- 2-1 – Event Storming: Thiết kế Hệ thống Đăng ký Tín chỉ (System Design [L2])
- 2-2 – Shared Library vs API — Khi nào nên gì? (Trade-off Analysis [L2])

2.9. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *SOA: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.8–10: Service-Oriented Analysis, Service Layers (task/entity/utility), Service Inventory Blueprint
2. [6] Eric Evans, *Domain-Driven Design* — Ch.1–5: Knowledge Crunching, Ubiquitous Language, Building Blocks; Ch.14: Bounded Context, Context Map
3. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.2: Decomposition Strategies
4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Loose Coupling, Dark Energy/Dark Matter Forces; Ch.7: Microservice Architecture; Ch.20: Assemblage Process
5. [4a] Sam Newman, *Building Microservices* — Ch.3: How to Model Services; Ch.10: Conway’s Law
6. [4b] Sam Newman, *Monolith to Microservices* — Ch.2: Planning a Migration, Decomposition by Domain
7. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.3: Service Boundaries, DDD/Bounded Context

Sách bổ trợ:

8. [9] Vaughn Vernon, *Implementing Domain-Driven Design* — Ch.2: Strategic Design with Bounded Contexts
9. [8] Matthew Skelton & Manuel Pais, *Team Topologies* — Four Team Types, Cognitive Load, Conway’s Law
10. [10] Alberto Brandolini, *Introducing EventStorming* — Domain discovery workshops

Nguồn trực tuyến:

- MacCormack, Rusnak & Baldwin, “*Exploring the Duality between Product and Organizational Architectures*” (2008) — Harvard Business School Working Paper

Tài liệu tham khảo

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.